



Tutorial: Using GitHub Through Posit Cloud

Karstens Lab
2026

Disclaimers

- This tutorial is meant to be an introduction. As such, there are more complex GitHub topics that we will not discuss in these slides.
 - For additional resources, see slides 36 and 37.
- This work builds off previous work completed by Karstens Lab researcher Benji Lamp.
- This tutorial will focus on using Posit Cloud, a free online interface.
- This tutorial was built to work with either OHSU secure GitHub or public GitHub.
- Example screenshots used in this slideshow show a specific GitHub repository, but the concepts apply to any repository you are using.

Tutorial Outline

- In this tutorial we will:
 - Introduce basic GitHub terminology.
 - Work through a short activity that goes through different steps of a common GitHub workflow:
 - Creating a Personal Access Token
 - Cloning a repository
 - Pushing your own code to a repo
 - Creating and merging pull requests
 - Pulling updates from GitHub back to R
 - Discuss how to leave feedback for others using GitHub

All of today's activities will focus on using Posit Cloud.

Prep Work: Installations

Pre-Tutorial Requirements

- To complete this activity, you must:
 - Have **Git** installed on your computer.
 - [Git Install Instructions \(Various OS\)](#)
 - Have a **Posit Cloud** account
 - [Posit Website](#)
 - Have a **GitHub** account
 - [How to Create a GitHub Account](#)
- Since we are using Posit, you do not need to have R or RStudio downloaded on your computer to complete the tutorial!

Check Git Install

- To make sure that Git installed properly, open your terminal and type the following:

`git --version`

- You should see some variation of the following message appear on your screen:

```
[(base) RNB16400:~ kudrick$ git --version  
git version 2.39.5 (Apple Git-154)  
(base) RNB16400:~ kudrick$ █
```

Background Information

What is GitHub?

- GitHub is a version control platform that allows you to:



Track the full history
of all project files



Revert a file to an
older version (if
necessary)



Easily collaborate on
code with others



Work on a project
across multiple
devices

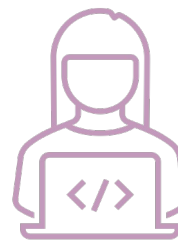
Why Use GitHub?

- Now that we understand what GitHub is, the next logical question is *why should we use it?*
- Some of GitHub's most helpful qualities are listed on the right.



Protects your files in case of a computer crash

Avoids having multiple files that cause confusion for your future self (or collaborators) in the future

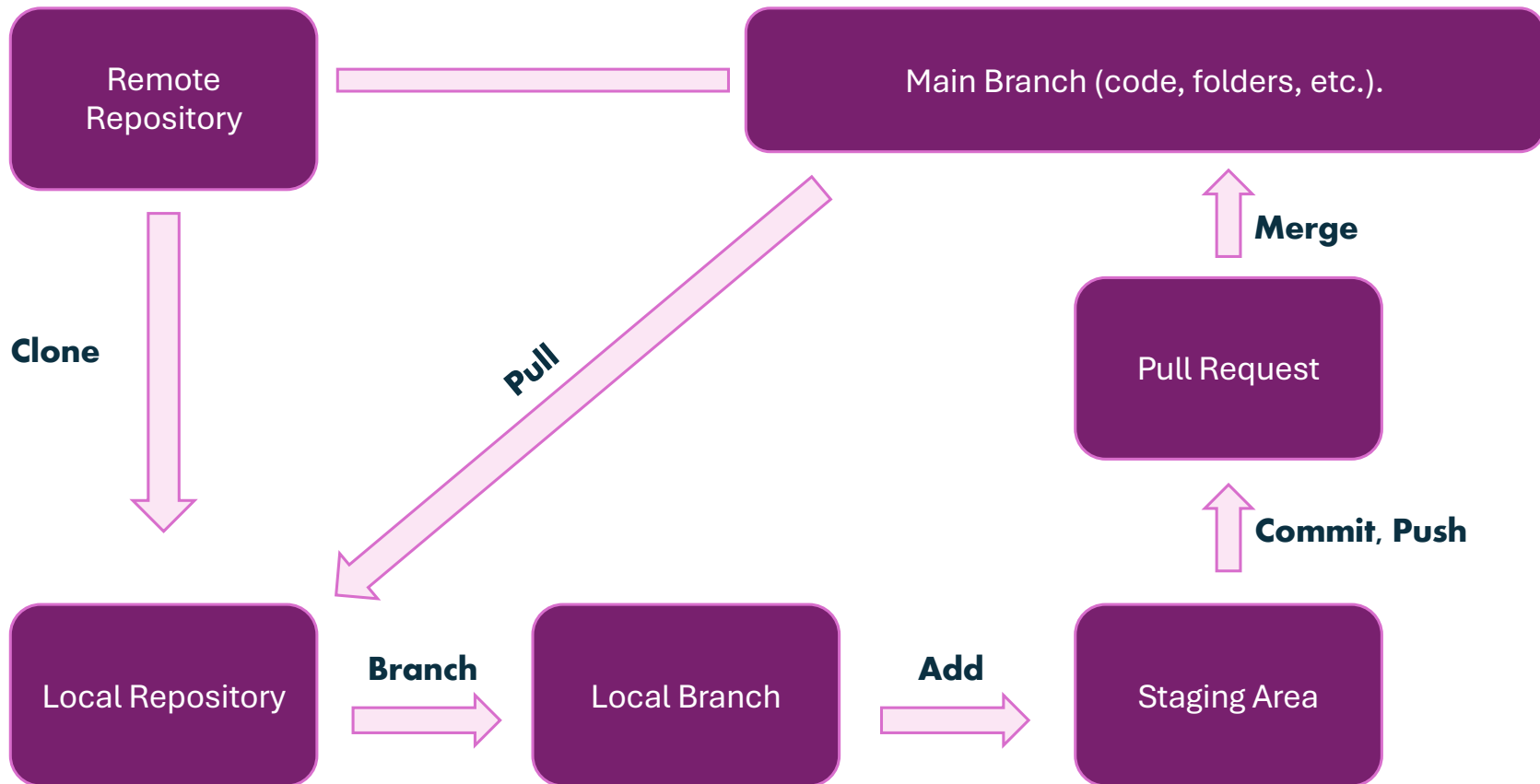


Allows you to experiment with your code, without needing to worry about permanently changing your files and/or causing downstream errors

Key Terms

Repository (Repo)	All files and folders associated with a given project. These live remotely on GitHub.
Clone	Creating a copy of a remote repository on your local device, allowing you to make edits locally.
Fork	Creating a remote copy of a repository that you can edit without impacting the original repo.
Branch	Creating a separate channel where you can complete edits without modifying the main code. This allows for multiple people to work on the same projects at the same time, without overwriting one another.
Commit	Telling Git to track the changes in a particular file
Push	Merging edits from your local device into the remote repository.
Pull	Syncing changes from the remote repository with your cloned version.
Issue	A way to report problems to the owner of a repository. Particularly helpful if using open-source packages.

GitHub Workflow: Overview



Set Up: Linking Posit Cloud with GitHub

Create a Personal Access Token (PAT)

- A PAT will allow you to use R and Git in collaboration:
 - Login to your GitHub account
 - Navigate to **Settings** → **Developer Settings** → **Personal access tokens/Tokens (classic)**
 - Click on **Generate new token**, and then **Generate new token (classic)**
 - Select **repo** and **workflow** for scopes
- Tips:
 - **Set a time limit for your PAT!**
 - Once you have generated your PAT, copy it immediately! It does not stay visible.

Settings / Developer Settings

Q Type 🔍 to search

All OHSU GitHub users must comply with OHSU's [Information Security](#), [Intellectual Property and Royalty Distribution](#), and [Acceptable Use of Computing and Telecommunications Resources](#) policies. GitHub users creating repositories under their own accounts are considered Data Stewards (see [definition](#)) of the information contained in their repositories. Link to [additional information](#) and [FAQ](#). Contact risxsupport@ohsu.edu for support during business hours.

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)

Generate new token ▼

Tokens you have generated that can be used to access the [GitHub API](#).

[Using Git and R for work projects.](#) — repo, workflow

Expires on Mon, Apr 6 2026.

Generate new token
Fine-grained, repo-scoped

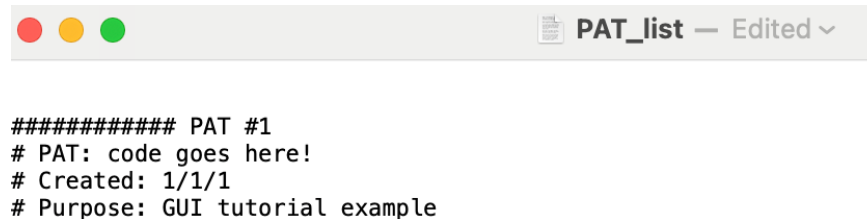
Generate new token (classic)
For general use

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Pro Tip: Save your PAT

- A PAT **will not remain visible** after you move into a new window.
- Thus, you must copy/save it somewhere safe, so that you can access it when necessary.
- Below, we provide an example approach for how to save your PAT for future access.

- Using Notepad (Windows)/TextEdit (MacOS) or the command line, create a plain .txt file.
- Copy your PAT into your file and save the file to a secure location on your laptop.
 - You can add more documentation if you'd like – whatever you feel is most helpful to you.
- An example of what a file could look like is shown below:

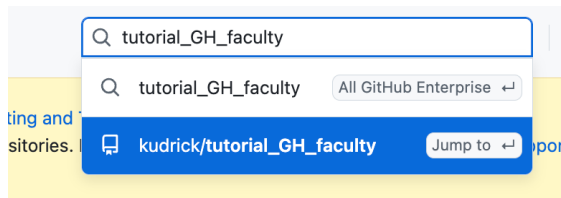


```
##### PAT #1
# PAT: code goes here!
# Created: 1/1/1
# Purpose: GUI tutorial example
```

Pro Tip: Returning to Repo

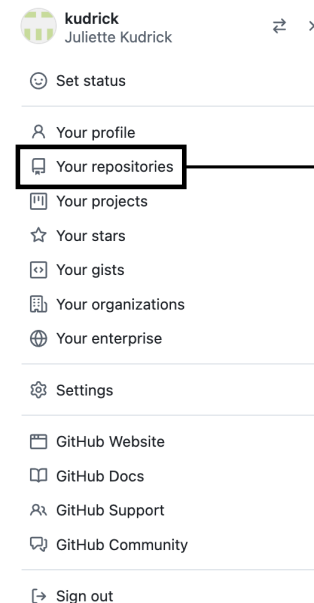
- To return to a repository after creating a PAT, there are a couple of approaches you can take.

Search Bar



If you know the name of the repository you are looking for, you can type it into the search bar at the top of the screen. The repo's name will pop up, and by clicking on it you can return to the repository.

Repository List



Clicking on “Your repositories” will redirect you to a list of all repositories you have created/are a collaborator on. This way, you can access the repository you are looking for, without needing to remember the exact name.

Authorize Posit Cloud

The screenshot shows the 'YOUR PERSONAL ACCOUNT' page for Juliette Kudrick. The left sidebar contains account details: Plan (Cloud Free), Current Usage Period (May 19, 2026 - Jun 19, 2026), Usage (Projects: 0 of 25, Compute hours: 0 of 25), and a list of menu items: Log Out, Profile, Authentication (highlighted with a red box), Credentials, and Account. The main content area shows the 'Email' section with the address kudrick@ohsu.edu and a 'Change Email' link. Below is the 'Login Options' section with a table of authentication methods:

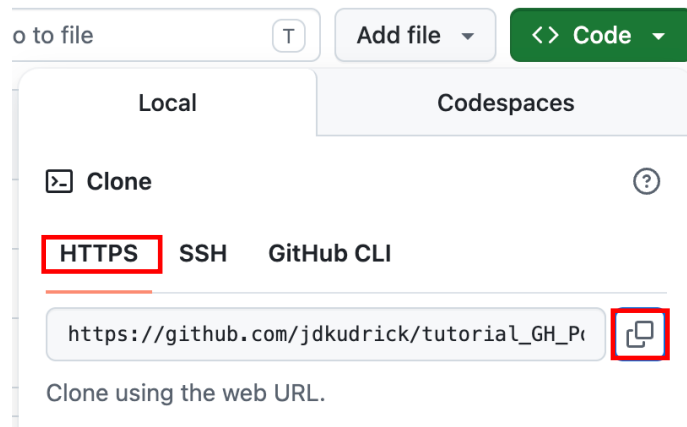
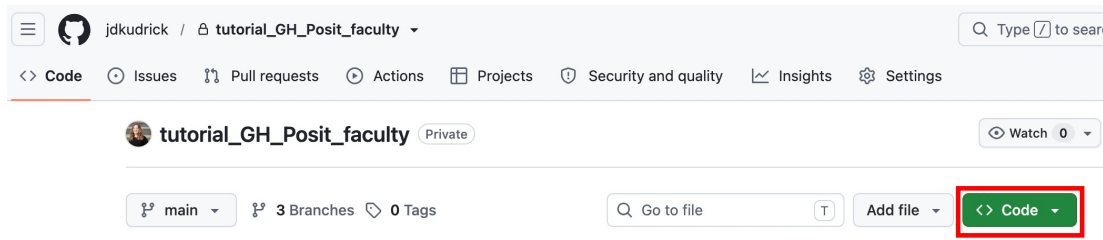
Login Option	Status	Additional Info	Action
Posit Password	☒ Enabled		Change Password
Google	☐ Enabled		
GitHub	☐ Enabled	☐ Private repo access also enabled (for Posit Cloud and shinyapps.io only)	
Clever	☐ Enabled		

Below the table is an 'Authorize Posit Cloud' dialog box. It shows the Posit Cloud logo and a GitHub logo with a green checkmark. The dialog contains a list of permissions: 'Posit Cloud by Posit Hosted Services wants to access your jdkudrick account', 'Repositories (Public and private)', and 'Personal user data (Email addresses (read-only))'. At the bottom, there are 'Cancel' and 'Authorize posit-hosted' buttons, with the latter highlighted by a red box. A note at the bottom states: 'Authorizing will redirect to https://login.posit.cloud'.

- Now, you must link your Posit Cloud account to your GitHub Account.
- Select **Authentication** → **Private repo access also enabled** → **Authorize posit-hosted**
- You should now be able to connect your two accounts.

Activity: Work Through Standard GitHub Workflow

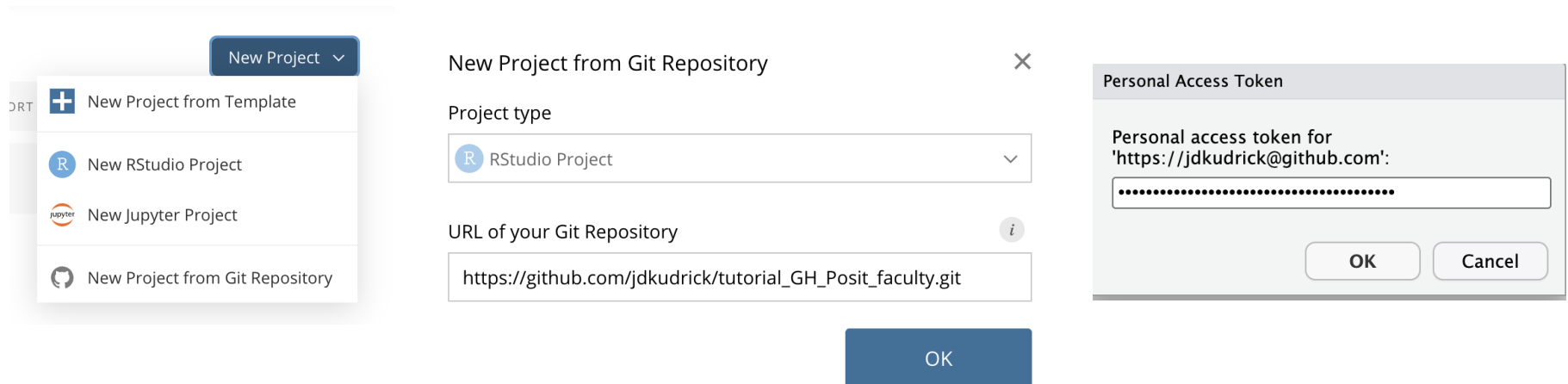
Step 0: Grabbing the Repo's URL



- On the repository's home page, select the drop-down arrow next to the button labelled **<> Code**.
- Making sure you are on the tab labelled **HTTPS**, select the **double square** icon next to the .git address. This will copy the repo's URL in preparation for cloning.

Step 1: Clone Your GitHub Repo

- Begin in Posit by clicking **New Project** → **New Project from Git Repository**
- When prompted, **paste the URL** for the repo
- Finally, when asked for a PAT, **copy your newly generated PAT** into the available space



Note: Be sure to enter your PAT and **NOT** your account password.

Overview: GitHub Commands via RStudio

The image shows the RStudio Git interface with several callout boxes explaining the functionality of different buttons and panels:

- Commit:** Allows you to commit changes you have made, preparing them to be pushed to the original repo.
- Pull:** Enables pulling from the original repo. This will sync the online repository with your cloned version.
- Push:** Here is where you can push commits you have made within your clone to the online repo.
- History:** Will show you the history of your project (when files were committed, who was the last person to edit the repo, etc.).
- New Branch:** This buttons allows you to create a separate branch from main. This enables more independent work.
- main:** Tells you which branch you are currently working in.
- Staged/Status:** **Staged** tells you if any changes have been committed. **Status** provides information on each file (has it been deleted, moved, renamed, etc.).

As you add/delete/edit files, changes will appear in this panel!

Note: If you change the width of the console in RStudio, the text labels shown above may disappear, leaving only the icons. Functionality remains the same.

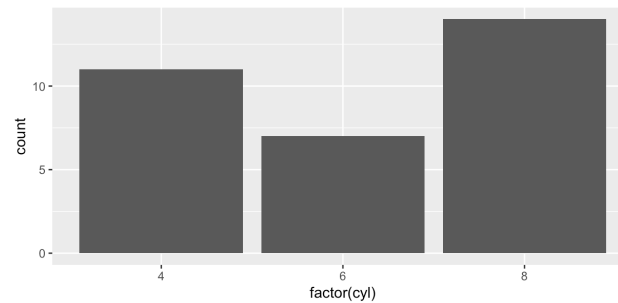
Step 2: Create a New .Rmd

- Create a new file and name it ***your_name_test.Rmd***.
- Create a code chunk and run the code below:

```
library(ggplot2)  
data(mtcars)
```

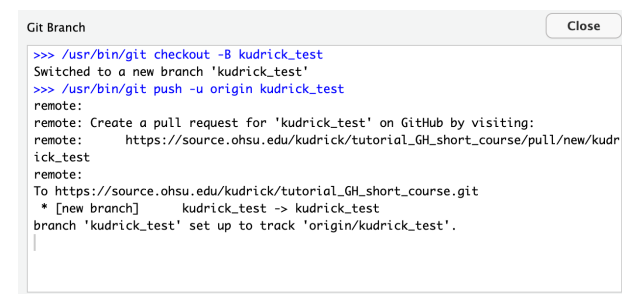
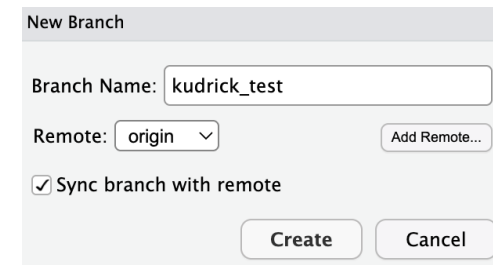
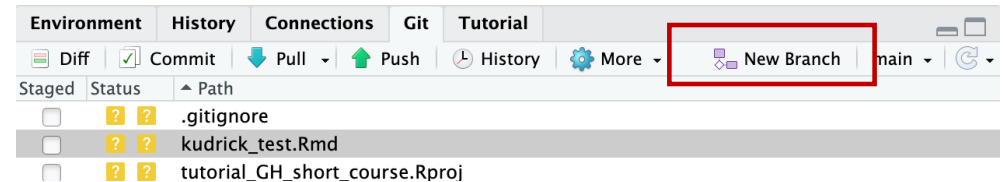
```
ggplot(mtcars, aes(x = factor(cyl))) +  
  geom_bar() +  
  xlab("Number of Cylinders") +  
  ylab ("Frequency")
```

- Running this code should produce the following plot:

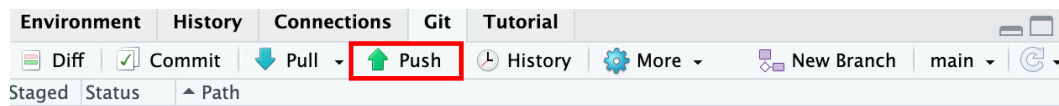
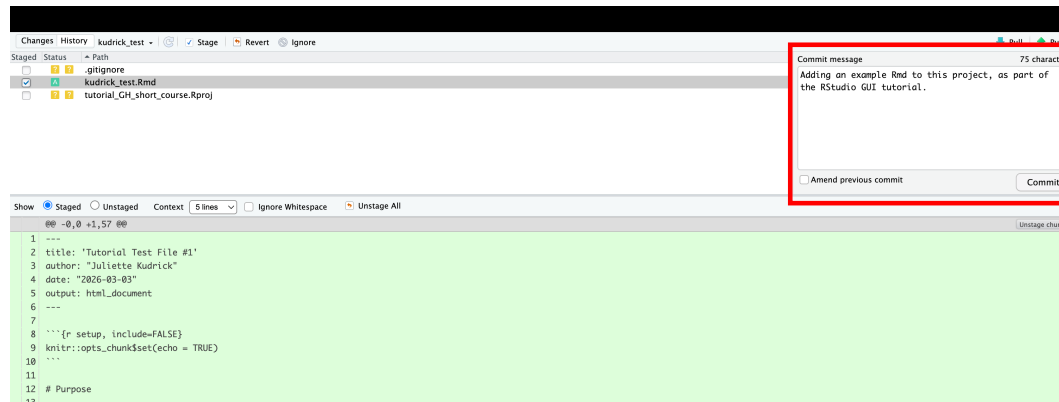
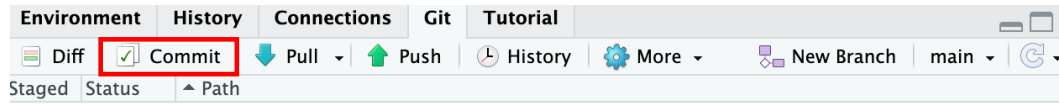


Step 3: Create a New Branch

- Next, we will create a new branch, separate from main, for our new script to live.
- Click on **New Branch**.
- Name your branch *your_name_test*
- Once you have created your branch, you should see a confirmation pop-up window.
- RStudio should automatically move you into your new branch. Confirm by looking at your current location, in the upper right corner.



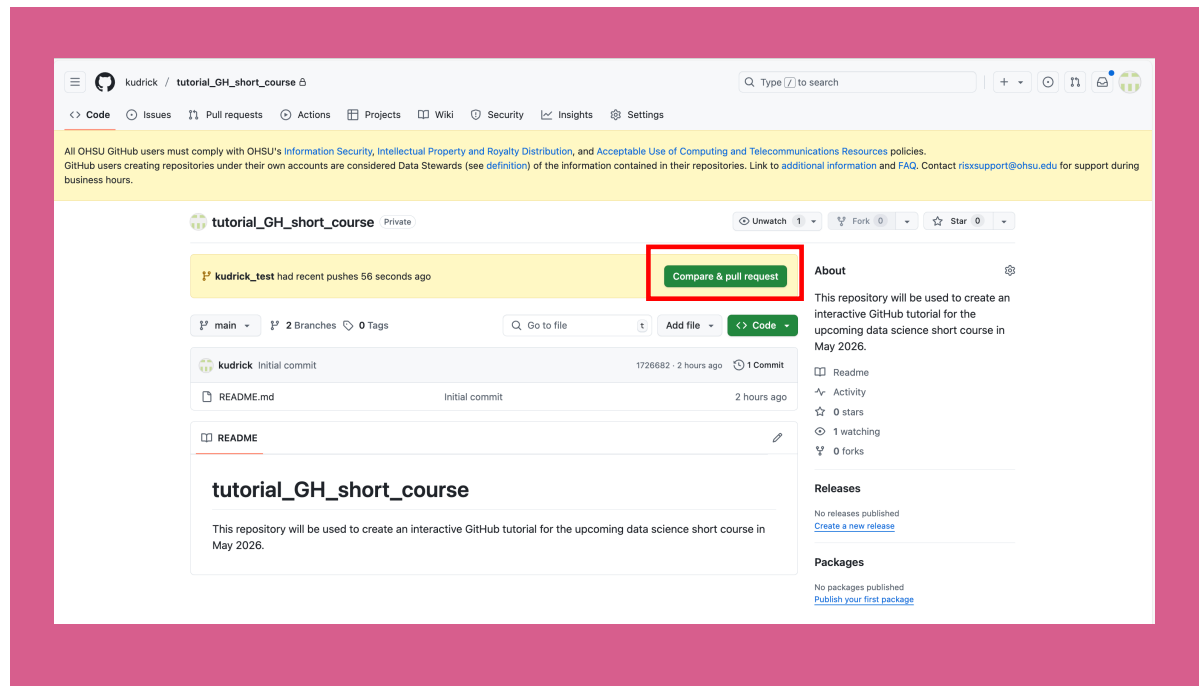
Step 4: Commit and Push Changes



- Now that we have created our .Rmd and moved into our new branch, we can commit our changes and push them to the GitHub repo.
- Click **Commit**.
- In the window that pops up, add a **commit message** explaining what you are doing, and then select Commit.
 - Note: commit messages should be succinct but detailed enough that collaborators and your future self understand what changes were made and why.
- Once this has been completed, exit out of the commit screen and select **Push**, to move your changes to the GitHub repo.

Check That Your Push Has Been Successful

- Once you have pushed your code to GitHub, you should see a “**Compare and pull request**” option on the GitHub website.
- This marks that your code has been successfully pushed and is ready for review.



Step 5: Create a Pull Request

- Click **Create pull request**
- You should see something like the image to the right.
- Assign your designated reviewer to your file.

Adding first test Rmd to tutorial repository #1

 Open kudrick wants to merge 1 commit into `main` from `kudrick_test` 

Conversation 0 Commits 1 Checks 0 Files changed 1

 kudrick commented now 

...orial.

 Adding an example Rmd to this project, as part of the RStudio GUI tut...  2b45372

 **Require approval from specific reviewers before merging**  Add rule 

[Rulesets](#) ensure specific people approve pull requests before they're merged.

 **This branch has no conflicts with the base branch**
Merging can be performed automatically.

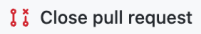
 Merge pull request  You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

 Add a comment

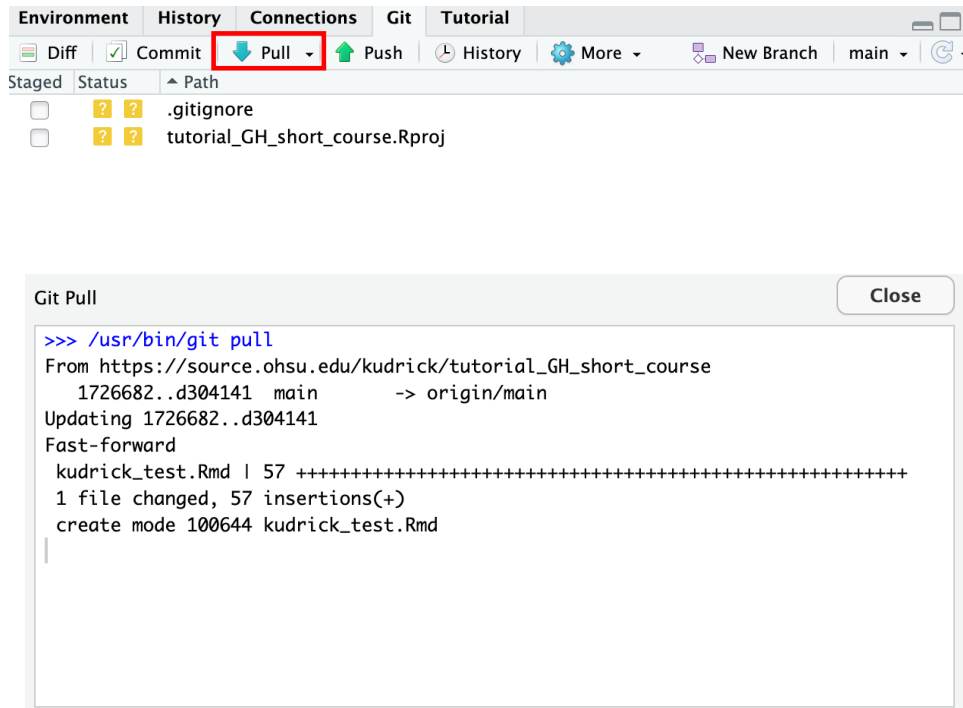
Write Preview             

Add your comment here...

 Markdown is supported  Paste, drop, or click to add files

 Close pull request  Comment

Step 6: Pull



The screenshot shows the RStudio Git interface. In the top toolbar, the 'Pull' button (a blue downward arrow) is highlighted with a red box. Below the toolbar, the 'Staged' and 'Status' tabs are visible, showing files like '.gitignore' and 'tutorial_GH_short_course.Rproj'. Below that, a 'Git Pull' dialog box is open, displaying the following text:

```
>>> /usr/bin/git pull
From https://source.ohsu.edu/kudrick/tutorial_GH_short_course
   1726682..d304141  main      -> origin/main
Updating 1726682..d304141
Fast-forward
 kudrick_test.Rmd | 57 +++++
 1 file changed, 57 insertions(+)
 create mode 100644 kudrick_test.Rmd
```

- After your reviewer has approved and merged your file into the main branch, return to RStudio.
- Click on the **Pull** button in the upper right-hand corner. This will sync your repo to your local clone.
- After pulling, you should receive a summary message explaining what changes were synced.
- With this, the activity is complete! You have now gone through a standard GitHub workflow.

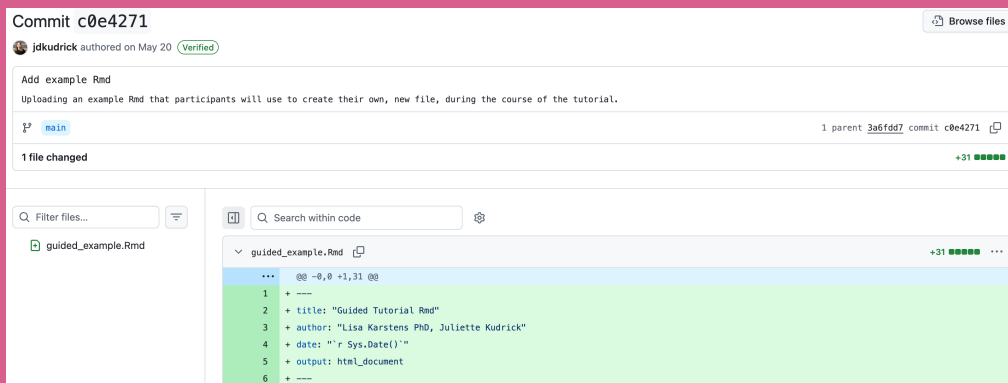
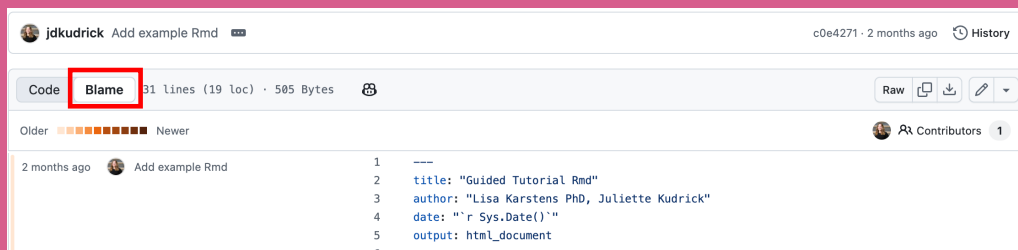
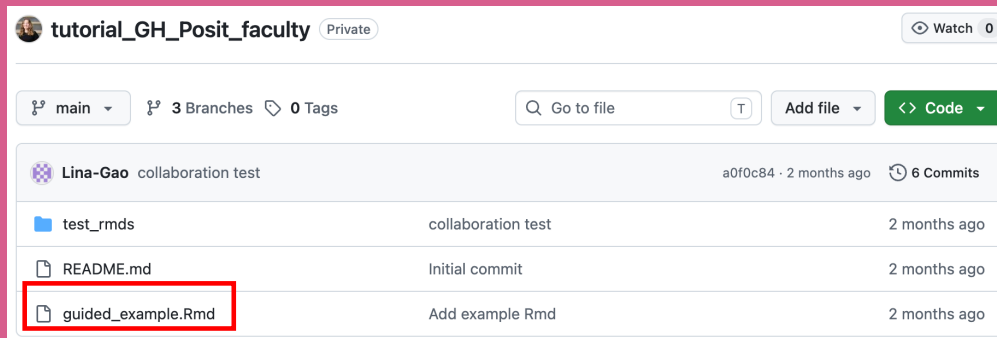
Bonus Content: Providing Feedback Through GitHub Reviews

GitHub Feedback

- In addition to its excellent version control functionality, GitHub also allows you to easily collaborate with others on code.
- In this bonus section, we will go through two approaches to providing feedback to other people's code.
 - 1) Leaving a review via comments
 - 2) Directly modifying code and proposing changes with branching + a pull request

Method 1: GitHub Reviews

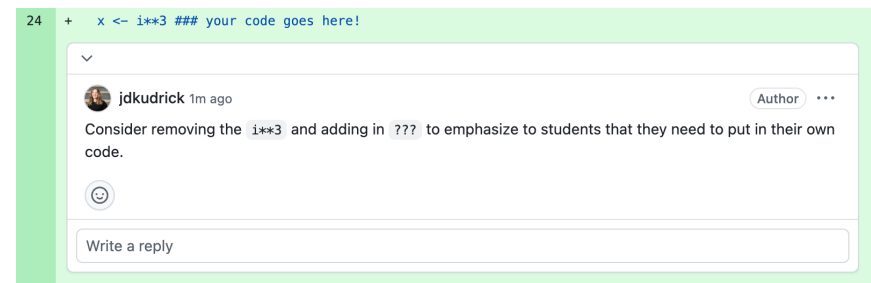
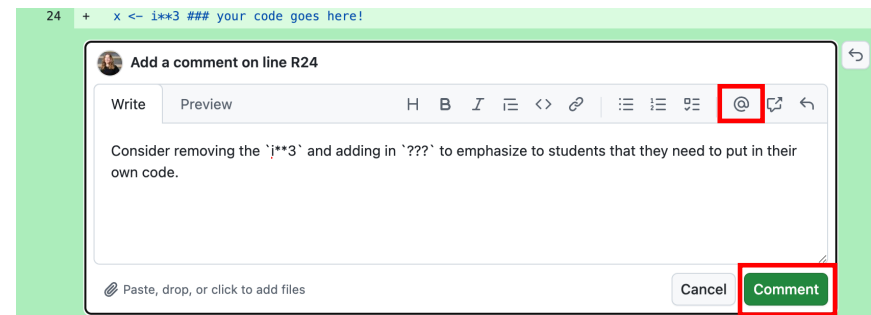
- GitHub reviews allow you to leave comments on someone's code without directly changing the scripts.
- This is very helpful for situations where you are functioning more as a reviewer than a hands-on collaborator.
- To leave a review, click on the file you wish to review → click **Blame** → select the most recent commit
- Within the commit, you can see the code, as well as who edited it and when.



Method 1 (Cont.)

- To leave feedback on a line of code, move your cursor to that line and select the **blue plus** that appears.
- Type your feedback in the comment box.
 - Comments support markdown formatting. To format code, encapsulate script suggestions in ``.
 - If you want to alert someone to a comment you have left, tag them using the @ icon.
- When you are done with your comment, select the green **Comment** button. An example of what a comment can look like is shown on the bottom right.

```
24 + x <- i**3 ### your code goes here!  
25 +  
26 + print(x)
```

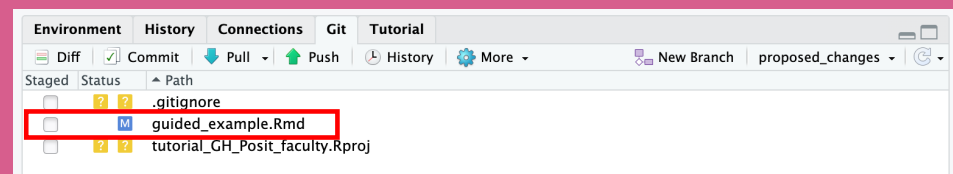
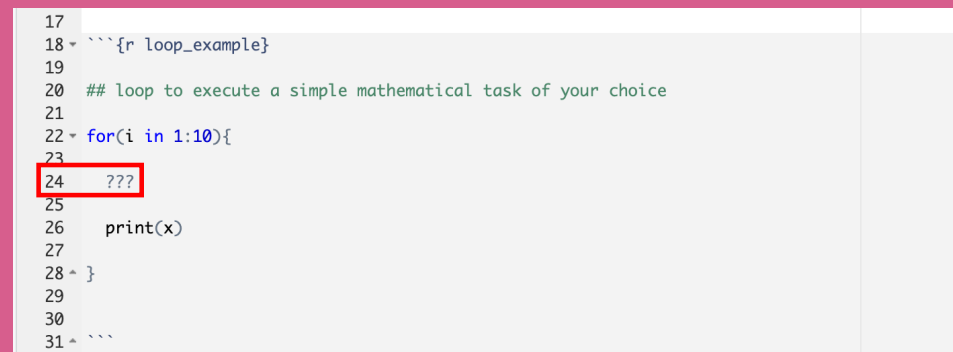
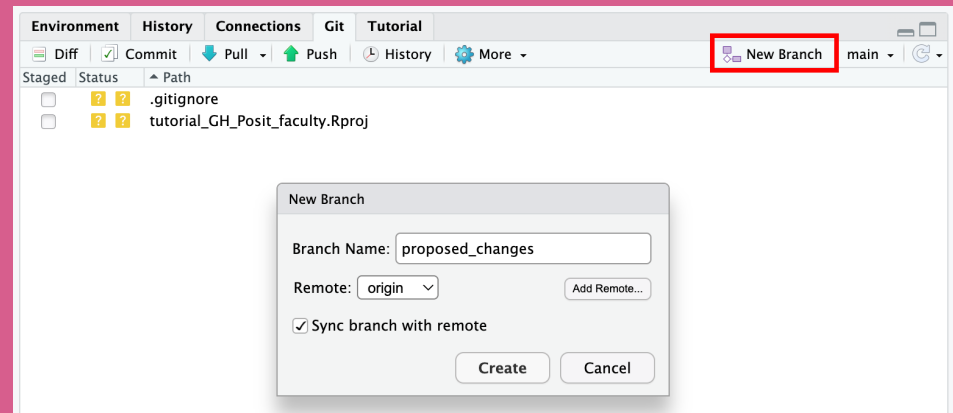


Method 2: Pull Requests

- Method 2 follows the same workflow that we worked through during our hands on activity.
- With this method, you will:
 - Clone someone's repository
 - Create a new branch
 - Modify someone's file
 - Push it to the repository and open a pull request.
- This method is preferred if you are actively collaborating with someone, with the goal of having them incorporate your proposed code directly into their project.

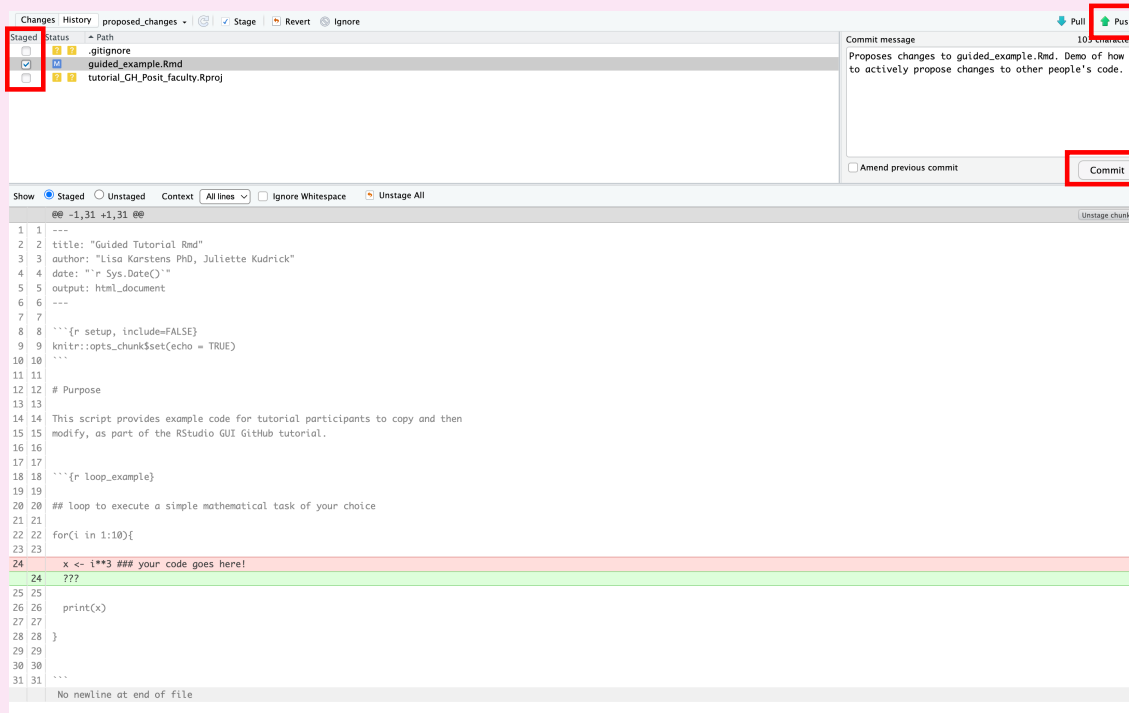
Method 2 (Cont.)

- For example: Let's say that I want to modify a file.
- Within my cloned repository, I will click on **New Branch** and create a branch titled `proposed_changes`.
- From inside this new branch, I will open and then modify `guided_example.Rmd`
- After I save the file, I can now see a **blue M** next to the file name, indicating it has been modified.

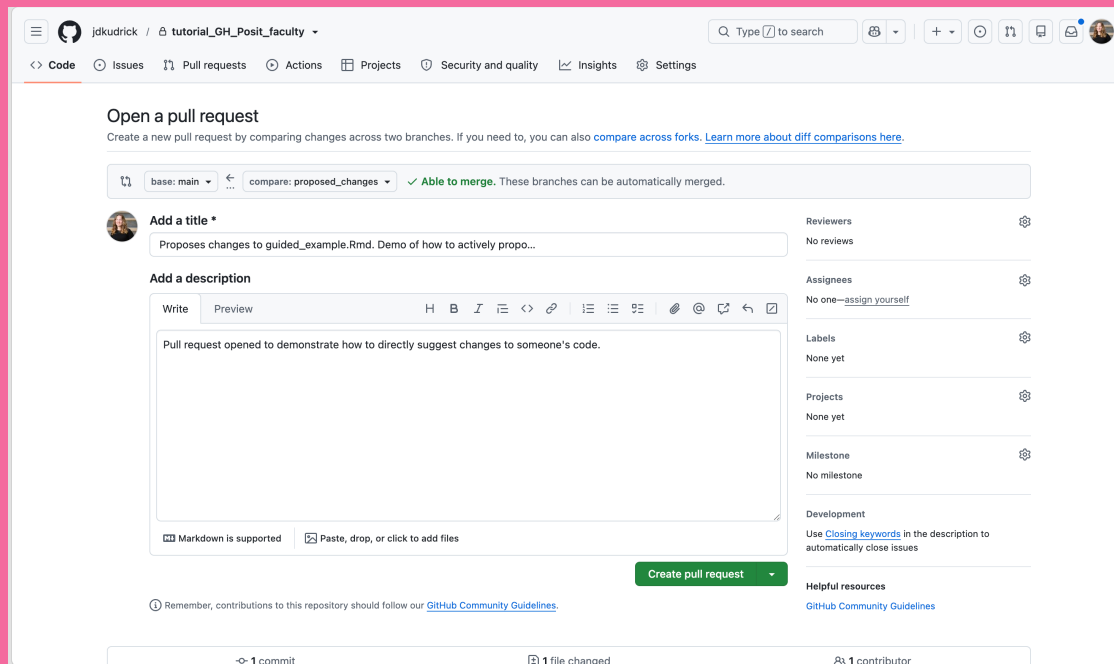


Method 2 (Cont.)

- Now that we have our modified file, we will go complete our previous workflow:
 - Stage
 - Commit
 - Push



Method 2 (Cont.)



The screenshot shows the GitHub 'Open a pull request' page for the repository 'tutorial_GH_Posit_faculty'. The interface includes a navigation bar with links to Code, Issues, Pull requests, Actions, Projects, Security and quality, Insights, and Settings. The main content area is titled 'Open a pull request' and provides instructions on how to create a new pull request. It features a 'base' dropdown set to 'main' and a 'compare' dropdown set to 'proposed_changes', with a green checkmark indicating 'Able to merge'. Below this, there are sections for 'Add a title', 'Add a description', and 'Add a review'. The 'Add a title' section has a text input field with the placeholder 'Proposes changes to guided_example.Rmd. Demo of how to actively propo...'. The 'Add a description' section has a text area with the placeholder 'Pull request opened to demonstrate how to directly suggest changes to someone's code.' and a 'Create pull request' button. The right sidebar contains sections for 'Reviewers', 'Assignees', 'Labels', 'Projects', 'Milestone', and 'Development'. The bottom of the page shows a summary of the pull request: '1 commit', '1 file changed', and '1 contributor'.

- Finally, we will open a pull request suggesting our changes.
- This allows whoever's code you have modified to review your changes and decide if they want to merge them or not.
- Any changes that are accepted will be merged into the main branch and will become the most up to date version of that particular file.

Additional Tips + Resources

Troubleshooting: Resources + Tips

- GitHub recommendations for:
 - [Cloning Errors](#)
 - [Authentication Errors](#) (General)
 - [Authentication errors](#) (Codespace)
- [Stack overflow](#) is a great resource
- Googling error messages can prove helpful
- Reach out to colleagues

Additional Resources

- [Git Documentation](#) (includes a cheat sheet + thorough documentation about Git and GitHub)
- [RStudio GUI Push/Pull Approach](#) (also includes helpful information about using GitHub and RStudio)
- [Forking v. Cloning](#) (different ways to copy repositories)
- [Additional Git Commands](#) (commands beyond basic ones covered here)
- [Git + RStudio Cheat Sheet](#) (an overview of the how to apply this workflow from the terminal).
- [GitHub CLI](#) (overview of something we did not cover here)
- [GitHub Scopes](#) (an explainer of different scopes and what permissions each one grants users)

If you have any questions, feel free to reach out: kudrick@ohsu.edu